Gene Ontology Analysis

An Advanced Programming Project

University of Bologna
Team members:

- *Luca Fratila*
- *Karen Cavina*
- *Tommaso Fabbri*
- *Natalia Nedyalkova*

Objective

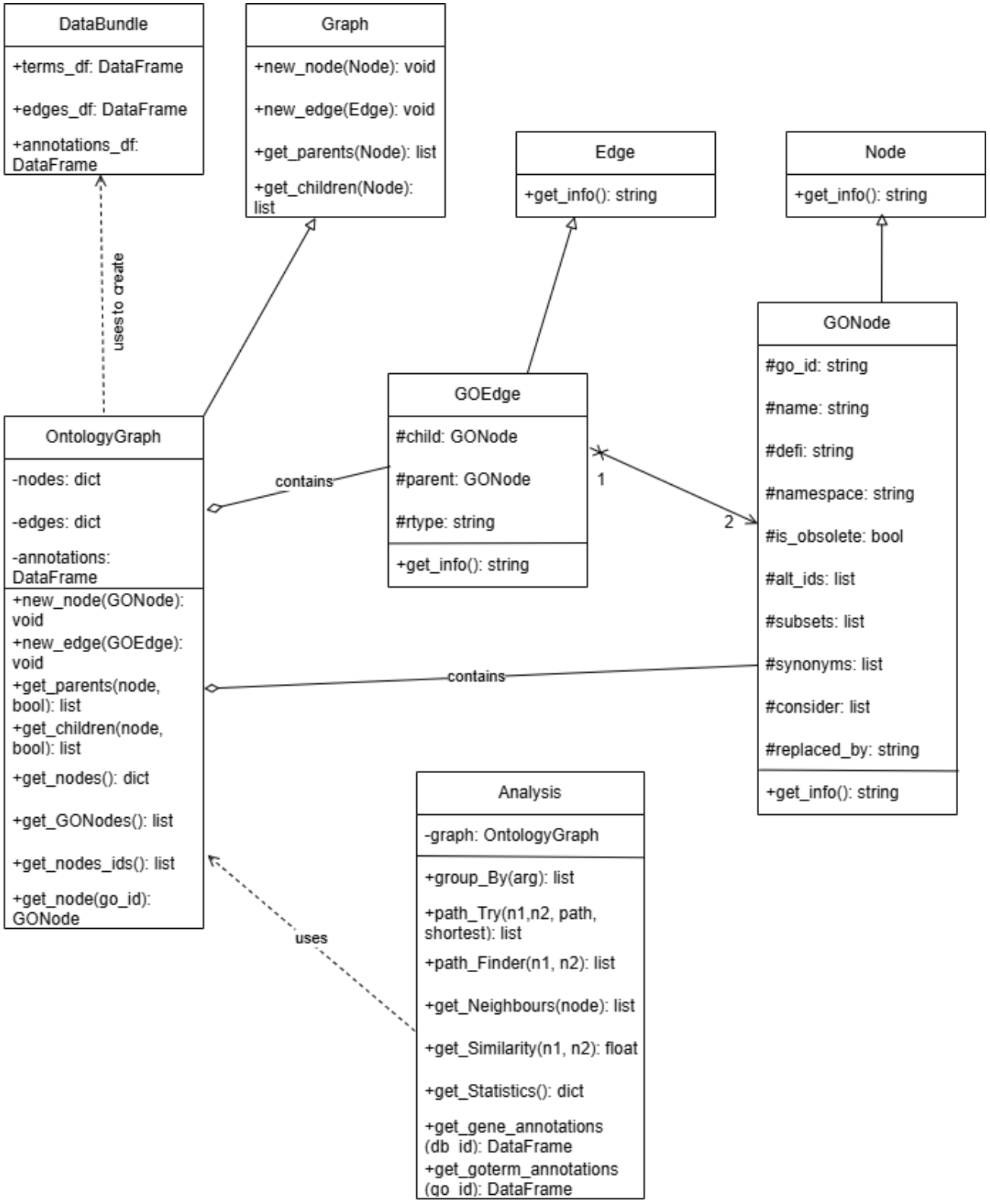
This project requires students to design and implement a modular, extensible software system for analysing the Gene Ontology and human gene annotations. Using the GO ontology (OBO format) and the GOA Human annotation file (GAF format), students must develop an object-oriented architecture that supports ontology representation, navigation, annotation analysis, numerical modelling, and web-based interaction.

The system must apply and demonstrate the four foundational principles of object-oriented programming through carefully designed components and behaviours.

Gene Ontology

Gene ontology is a framework whose purpose is to describe and define several characteristics of a collection of genes in a genome and their products. Split between the ontology and the annotations of the individual genes, such characteristics include but are not limited to GO (gene ontology) IDs, GO terms, and product information. Through analysis of the data stored in gene ontology databases, we may identify several biological processes and functions with the help of programmes across a variety of programming languages such as R or Python.

UML class diagram



CRC cards

DataBundle	Superclass	Subclass
	N/A	N/A
Responsibilities		**Collaborations**
Store parsed datasets by Data Module		OntologyGraph
Provide access to terms_df, edges_df, annotations_df		
Keep outputs grouped together		

Graph	Superclass	Subclass
	ABC	OntologyGraph
Responsibilities		**Collaborations**
Create graph nodes		
Create graph edges		
Identify parents/children		

Node	Superclass	Subclass
	ABC	GONode
Responsibilities		**Collaborations**
Represent Node in Graph		
Define what a node is		
Allow access to specific Node information		

Edge	Superclass	Subclass
	ABC	GOEdge
Responsibilities		**Collaborations**
Represent Edge in Graph		
Define what an edge is		
Allow access to specific relationship information		

GONode	Superclass	Subclass
	Node	N/A
Responsibilities		**Collaborations**
Store go_id and other information about the Node		GOEdge
Provide information about the term		OntologyGraph
Ensure validity of name		Analysis

GOEdge	Superclass	Subclass
	Edge	N/A
Responsibilities		Collaborations
Store child&parent terms		GONode
Store relationship type and provide information		OntologyGraph
Ensure both objects are nodes		

OntologyGraph	Superclass	Subclass
	Graph	N/A
Responsibilities		Collaborations
Create ontology graph using GONodes and GOEdges		GONode
Add new nodes and edges to graph		GOEdge
Store annotation data for nodes		DataBundle
Retrieve parent&child terms and other information about nodes		Analysis

Analysis	Superclass	Subclass
	N/A	N/A
Responsibilities		Collaborations
Analyse an OntologyGraph		GONode
Group nodes by namespace		OntologyGraph
Find paths between nodes and adjacent nodes		
Compute similarity score between nodes and graph statistics		
Retrieve annotations by gene id or GONode id		

Object oriented programming

Object Oriented Programming is, by en large, a way of structuring code by focusing on so-called ‘objects’: combinations of attributes and behaviours. The usefulness of OOP truly shines when it comes to larger projects, especially ones that deal with multiple modules, such as this one.

Due to the sheer complexity of these projects, disorganisation and difficulty of use typically scale exponentially if they are not organised properly. Yet, by following an OOP structure, the organisation, ease of use, and maintenance of such projects are preserved even as the number of modules increases.

The four properties of Object Oriented Programming are:

1) **Encapsulation**
   Encapsulation refers to the organisation of the code such that the behaviour of the object is defined within the object itself, removing the need for external calls for manipulation.

2) **Abstraction**
   While the actual mechanisms and behaviours that an object may possess, there must be an emphasis on ease of use. As such, while the object itself may be complicated, calling it must be as easy as possible in order to accommodate the users of the program.

3) **Inheritance**
   By allowing classes to 'inherit' information or behaviours from preceding classes, the number of lines of code can be drastically reduced, which, in larger projects, may even help with space and time optimisation.

4) **Polymorphism**
   This property allows for a large degree of flexibility and 'independence', as multiple objects can provide different outputs when called by the same command, reducing the need for an incessant number of methods.

Code Architecture

I: Parser and data handler

The main role and function of this first module is to read through the files of a gene ontology database and extract the relevant information based on their annotations. After defining a simple `DataBundle` class and ensuring that the program can read both zipped and unzipped files, we transform the data from the gene annotation file into a pandas dataframe. The reasoning behind using pandas can be boiled down to efficiency and ease of use regarding how the data should be stored in order for it to be easily accessed by the subsequent modules.

By creating columns for each of the annotation terms, such as:

- "go_id"
- "taxon"
- "qualifier"

We are able to extract and store the annotation information using a for loop that iterates over each line in the annotations file, adding the information in the form of rows for

each gene. Once the table storing the genes' data is completed, the program must produce 2 separate data frames containing GO terms and relationship information that is required for analysis in future modules. By reading over the data selectively, we are able to complete two dictionaries, which are then transformed into pandas data frames for simplicity.

After a brief 'sanity check' to make sure that the program does not perform wrong operations by checking for duplicate IDs, that all child IDs are valid GO IDs and that each annotation is paired with a valid GO ID, the program ends with the function that ties it all together by calling the previously defined functions which:

1) Load and validate the information from the GO files
2) Go through and analyze the ontology and annotations
3) Validate the results again if requested by the user
4) Create the databundle, ready for use in analysis and/or graphing

The use of Object Oriented Programming in this module is straightforward – the `DataBundle` class reduces the complexity and increases the ease of use when called in subsequent modules through the previously defined OOP properties. The function groups results together and is easy to call. Due to the objectives of the module, the `DataBundle` class did not require resorting to polymorphism and inheritance, yet it retains the benefits of Object Oriented Programming.

II: Ontology representation

The graph module has the objective of creating an intuitive and useful object representation of the data extracted in the first module. Data is stored as an acyclic directed graph, which is a set of objects called nodes connected by a set of edges, and these edges are clearly exiting from one node and entering into another (hence the directionality). The module makes use of many core properties of OOP, mainly abstraction and polymorphism.
Some core classes are used for this purpose: `GONode` to store information about a single GO term, `GOEdge` to store information about the relationship between two GONodes, and `OntologyGraph` that uses the aforementioned classes and builds the corresponding graph. Each of these specialized classes is a subclass of an abstract class (Node, Edge, Graph), which means the user could use them to build any kind of graph that is not specifically related to gene ontology, so the code is extensible.
The user can use the function `create_graph` which, given a `DataBundle` object in input, will return a `OntologyGraph` object with the same information but structured as an acyclic directed graph. This function will iterate through every row of the dataframes present inside a `DataBundle` and create the right kind of object (`GONode` or `GOEdge`). The annotations are not iterated through but just directly fed to the new `OntologyGraph` object as `DataFrame`, simply because of the big number of rows which would have stopped the program for more than an hour, which is not a sacrifice we are willing to make since the information needed for analysis can be easily extracted from the dataframe.
An `OntologyGraph` object is structured in such a way:

- `self.__nodes` is a dictionary where each key corresponds to the go_id string of a GO term and the value is the GONode object associated with that go_id, making it easy to search for a term even if we only know its go_id.
- `self.__edges` is another dictionary where each key corresponds to the go_id string of a GO term and the value is a list of edges ending into that GO term (basically a list of all edges where the GO term is the child); This corresponds to an adjacency list representation of a graph. The dictionary structure proved to be the best to access the graph contents in a fast and easy way.
- `self.__annotations` is a DataFrame with all the information extracted from the GAF file.
- Some useful methods are defined inside the class itself, which will be useful for any kind of analysis, such as `get_parents` and `get_children`, plus some useful methods to obtain the list of nodes present in the graph since the property itself is made private to protect from external manipulation.

Additionally, we exploited polymorphism inside `GONode` and `GOEdge`: both classes have a `get_info()` function which returns a string with the most useful information based on the kind of object.

III: Analysis and comparison of data

The third module analyzes the graph generated in the previous steps through the Analysis class, which is initialized by taking a OntologyGraph object in input. Inside this class there are a variety of methods that analyze the components of the files that the user fed to our program:

- `group_By` returns a list of all the nodes that belong to a specific namespace;
- `path_Finder` exploits the graph structure of the data by calling in both directions `path_Try` which will then find recursively the shortest path between two nodes;
- `get_neighbours` returns a list of all the parents and children of a given node;
- `get_similarity` evaluates the similarity between two nodes using the Dice-Sørensen coefficient statistic: the idea is to keep track of the total amount of properties between the two nodes (the variable `n1N2Prop`) and computing the number of properties that are either shared or somewhat similar (denoted by the variable `overlap`). The final value of overlap is found with the use of three encapsulated functions: `propertyCompareStr`, `propertyCompareList` and `propertyCompareId`, each one being used to check the properties of the specific data type and returning a float value to be added to `overlap`. The final similarity is a float computed by dividing overlap by `n1N2Prop`;
- `get_statistics` compiles a dictionary of various useful statistics regarding the graph nodes, such as average neighbour amount, number of obsolete terms and a list of the nodes with the most synonyms among the others. Due to technical limitations a complete dictionary normally obtainable from the function can be called directly, as the execution time of the function is in the range of hours and we expect the program to only work on the files provided with the assignment.

IV: User interface

The role of this last module is to create a web application and manage its interface. We chose Flask because it allows us to create a link between the functions implemented in the other modules and a browser-based interface without having to rewrite them. In this way, the module acts as the controller of the application, receiving requests from the user, calling the appropriate functions, and passing the results to the webpages for display.

After this, we created a dictionary called `STATE`, which stores the uploaded file paths, parsed datasets, ontology graph and analysis object after upload. `STATE` also allows us to limit access to certain pages of the website depending on the state of the application. For example, statistics and ontology searches cannot be performed before the OBO and GAF files have been uploaded and parsed successfully.

To ensure that the uploaded files are valid, we created a simple function `allowed_file`, which checks that the uploaded files are of an appropriate type, and only accepts: .obo, .gaf, .gaf.gz files. We also used a Flask error handler and size limit to ensure that the uploaded files are not excessively large. This prevents the application from crashing and instead redirects the user back to the upload page with an appropriate error message.

The next step was to create the routes of the application. We did that in such a way that each route connects to a specific HTML and its own Jinja template. We chose Jinja templates because they are a built-in Flask's templating system that allows us to create a link between the Python data and the HTML webpages. This enables us to display ontology terms, relationships, statistics, similarity scores and gene annotation information directly into the browser.

The implemented routes allow the user to upload ontology and annotation datasets, search for GO terms, inspect parent and child relationships between terms, compute similarity scores between different nodes and calculate statistical information about the ontology graph.

Conclusion

The different modules of the project are able to work together, employing each other's functions in order to achieve our goal, that is to provide the user with a simple interface that is able to load, parse and analyze GO and GAF files. In module 1 we take in input the files and store important information inside dataframes that are fed to module 2, in order to build a graph where each GO term is a node and their relationships are translated into edges. Module 3 is able to compute similarity scores and various types of analyses using the graph object created in the previous steps. Finally, module 4 is needed to actually launch the web application and makes calls to the functions built in the other modules to provide a satisfactory experience for the user.